

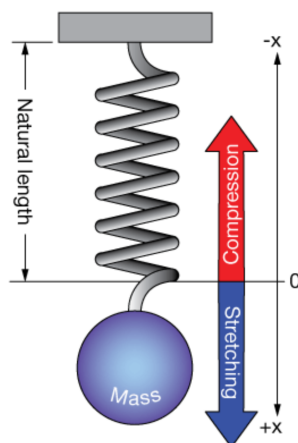
# MAF220 — Numerical Modelling

Computer Lab, Fall 2026

In the first lecture, we installed Anaconda and wrote our first Python program with Jupyter. Today we will learn about NumPy and its linear algebra functionality.

## Exercise 2:

Let's say we have a weight on a spring: If the system is displaced from its equilibrium position, it will oscillate about its natural length. The system is equipped with a measurement device that records the position of the mass relative to the natural length of the spring as a function of time.



We are given some data from this measurement device when the system was oscillating. Unfortunately, the device only recorded 8 data points. Given these 8 data points, we would like to reconstruct the trajectory of the mass! We will use a method of linear algebra since you are probably eager to try out some of the things you learned in MAF210.

Here are the data points:

$$t = [0.3, 0.36, 0.94, 0.96, 0.98, 1.8, 1.86, 1.98] \text{ s}$$

$$x = [0.81, -1.02, -0.05, -0.75, -1.09, -1.02, 0.08, 1.04] \text{ cm}$$

In this exercise, we are going to fit a polynomial to these data points to explore some numerical methods. Consider a polynomial of degree  $N$ :

$$p(x) = c_0 + c_1 \cdot x + c_2 \cdot x^2 + \cdots + c_N \cdot x^N$$

The objective of this exercise is to fit this polynomial to  $N + 1$  data pairs  $(x_i, y_i)$ . We, therefore, want to determine the  $N + 1$  coefficients  $c_i$  of the polynomial to satisfy the following  $N + 1$  equations:

$$\begin{aligned} p(x_0) &= y_0 \\ p(x_1) &= y_1 \\ p(x_2) &= y_2 \\ &\vdots \\ p(x_N) &= y_N \end{aligned}$$

These can be rewritten in terms of a linear system  $\mathbf{A}\mathbf{c} = \mathbf{y}$ , where  $A$  is the Vandermonde matrix. Explicitly:

$$\begin{pmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^N \\ 1 & x_1 & x_1^2 & \cdots & x_1^N \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_N & x_N^2 & \cdots & x_N^N \end{pmatrix} \begin{pmatrix} c_0 \\ c_1 \\ \vdots \\ c_N \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_N \end{pmatrix}$$

To construct the Vandermonde matrix with for loops, it is helpful to realize that its entries are given by  $a_{ij} = x_j^i$ . To construct this matrix and make our way to a solution we will be proceeding step by step:

1. Create a list containing the  $t$  values and a list containing the  $x$  values.
2. Plot the data points with Matplotlib (label the axes and use a scatter plot with `plt.scatter(...)` since we do not want the data points to be connected).
3. Set up the Vandermonde matrix  $A$  (use two nested for loops) and the vector  $\mathbf{y}$ . (Replace in the formulas above  $x \rightarrow t$  and  $y \rightarrow x$ .)
4. Solve the matrix equation with NumPy (Note that it is more efficient to use the `solve` method of NumPy instead of calculating the inverse of the matrix). What are the values of the coefficients?
5. Create a function that evaluates the polynomial given the coefficients.
6. Create an array with 500 entries for  $t$ -values ranging from the lowest value to the maximum value of the  $t$  data list. Create a second array with 500 entries that contains the values of the polynomial evaluated at the 500 element  $t$ -value list.
7. Plot the original data points and the data points of your fit in one plot. Does the line of your fit go through the original data points?

**Interpolation vs extrapolation:** Now we have fit a polynomial to our data. Looks pretty good doesn't it? However, we've only plotted our result within our fitting-region. Let's have a closer look at the obtained polynomial:

8. Repeat the last two steps, but this time extend the range of the  $t$ -values to 0 and 2.5. How does the fitted function behave outside the range of the fitted data points?

**Robustness of the fit:** We have seen that we can only interpolate our data but not extrapolate it. But is it robust, i.e. stable under small perturbations?

9. After some time, you realize that the measurement device does not accurately measure position. It is very accurate in time measurements, though. The uncertainty for position measurements is  $\pm 0.1$  cm.
10. Let's see how the fitted polynomial changes if we change data point 4 by  $x[3] = x[3] + 0.1$ .
11. Plot both polynomials in one plot. Given the tiny change in the data points, are the changes in the polynomial small or large?
12. Indeed, the changes in the polynomial are surprisingly large. This is generally the case for matrix equations where the matrix  $A$  has a large condition number. It quantifies how small changes in a vector  $\mathbf{c}$  affect the result of  $\mathbf{A}\mathbf{c}$ . Use this NumPy function to calculate the condition number of the Vandermonde matrix: `t=np.linalg.cond(A)`.

**Extra:** NumPy provides the `polyfit` method, which allows you to fit a polynomial to data points without setting up the Vandermonde matrix. Look up the documentation and use this method to fit and evaluate (use `polyeval`) polynomials. Recall 8 data points  $\rightarrow$  polynomial degree 7.

**Closing remarks:** The exercise is mainly about applying linear algebra. The results of this fit method are not robust, as we have seen. We can obtain better results by adding some physics knowledge. We will come back to this in a few weeks!